HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO BÀI TẬP LỚN

Môn: Mô phỏng hệ thống truyền thông

Hà Nội, 2023

MỤC LỤC

1 Nhiệm vụ 1: Mô phỏng quá trình PCM cho tín hiệu tương tự 3

1.1 Mô tả nhiệm vụ 3

1.2 Mô tả hệ thống mô phỏng và các tham số 4

1.3 Kết quả: 6

1.4 Mã chương trình 8

2

3. Nhiệm vụ 3

1 Nhiệm vụ 1: Mô phỏng qua trình PCM cho tín hiệu tương tự

I. Nhiệm vụ 1

1. Mô tả nhiệm vụ:

Mô phỏng quá trình PCM cho tín hiệu tương tự, trong đó tín hiệu tương tự dạng thoại đầu vào được mô tả bởi công thức sau:

$$s(t) = \sum_{i=1}^{n} A_i cos(2\pi f_i t + \varphi_i)$$

trong đó, n là số sinh viên trong nhóm, φ_i lần lượt là $[0, \pi/2^1, \pi/2^2, \ldots, \pi/2^{n-1}]$ và nếu gọi $maso_i$ ($i = 1,.., n$) lần lượt là n số cuối trong mã số của sinh viên nhóm trưởng thì $A_i = maso_i$ và tần số f_i được xác định theo biểu thức $f_i = maso_i$ x100 Hz.

Thực hiện chuyển đổi tín hiệu âm thoại yêu cầu trên thành tín hiệu số nhị phân theo luật A tiêu chuẩn châu Âu để làm nguồn tin đầu vào cho nhiệm vụ 3 và thực hiện ngược lại chuyển đổi chuỗi bít nhị phân thu được thành tín hiệu tương tự dạng âm thoại.

2. Yêu cầu:

a. Mô tả hệ thống PCM chuyển đổi tín hiệu yêu cầu bằng sơ đồ khối và xác định các tham số của hệ thống.

b. Bằng việc sử dụng MATLAB, viết chương trình và thực hiện mô phỏng quá trình PCM theo tiêu chuẩn châu Âu cho tín hiệu thoại sử dụng kỹ thuật nén – dãn theo luật A với A = 87,6 và tần số lấy mẫu f_s = 8kHz.

c. Biểu diễn dạng sóng tín hiệu trên khoảng 3 chu kỳ và phổ của nó tại các điểm sau trên hệ thống: tín hiệu gốc bản tin và tín hiệu được khôi phục tại đầu thu hệ thống PCM.

1.1 Mô tả nhiệm vụ

- Mô phỏng quá trình PCM cho tín hiệu tương tự s(t):

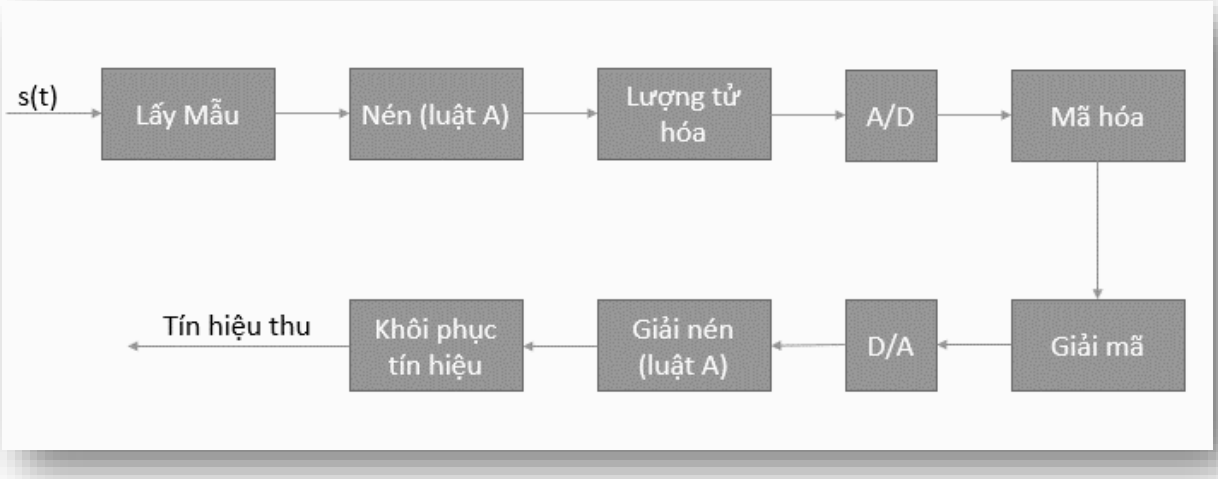
$$s(t) = \sum_{i=1}^{3} A_i \cos(2\pi f_i t + \varphi_i)$$

với MSV: B20DCVT227

s(t)= 2*cos(2*pi*200*t)+2*cos(2*pi*200*t+pi/2)+7*cos(2*pi*700*t+pi/4)

- Thực hiện chuyển đổi tín hiệu tương tự thành tín hiệu nhị phân và ngược lại với tiêu chuẩn nén và giải nén luật A

1.2 Mô tả hệ thống mô phỏng và các tham số



Hình 1.1: Hệ thống mã hóa và giải mã PCM sử dụng trong mô phỏng

Mô tả hệ thống và các bước tiến hành mô phỏng:

Lấy mẫu:

- Lấy mẫu là quá trình biến đổi tín hiệu tương tự thành một dãy xung rời rạc tuần hoàn, biên độ xung bằng biên độ tín hiệu tại thời điểm lấy mẫu. Dãy xung này còn được gọi là tín hiệu điều chế biên độ xung PAM. Tín hiệu tương tự được lấy mẫu bằng cách nhân tín hiệu với chuỗi xung răng lược có chu kỳ bằng chu kỳ lấy mẫu
- Mô phỏng: Tạo vecto thời gian bắt đầu từ thời điểm 0 đến thời điểm kết thúc tín hiệu tương tự (T), vecto gồm các phần tử cách đều nhau một khoảng bằng chu kỳ xung răng lược: t=0:Ts:T. Thay giá trị vecto thời gian vào tín hiệu tương tự, ta được một vecto chứa biên độ các xung PAM
- Tham số sử dụng. Tín hiệu tương tự được lấy mẫu với tần số fs=8e3 Hz nên chu kỳ lấy mẫu Ts=1/fs=1.25e-4 s

Nén luật A:

- Bộ nén là một bộ khuếch đại phi tuyến, mục đích của bộ nén này là tạo ra các mức lượng tử hóa không đều nhằm cải thiện tỷ số tín hiệu trên tạp âm S/N. Có hai phương pháp nén thông dụng là nén luật μ và nén luật A. Phương pháp nén luật A như sau: gọi tín hiệu s1(t) là tín hiệu vào bộ nén, tín hiệu s2(t) là tín hiệu ra ta có mối quan hệ

$$|s2(t)|=\begin{cases} \dfrac{A.|s1(t)|}{1+\ln(A)} & nếu\ 0 \le |s1(t)| \le 1/A \\ \dfrac{1+\ln(A.|s1(t)|)}{1+\ln(A)} & nếu\ 1/A \le |s1(t)| \le 1 \end{cases}$$

- Mô phỏng: Nén luật A có thể thực hiên bằng hàm compand(s,A,max(s),'A/compressor') với s là vecto tín hiệu sau lấy mẫu, A là hằng số nén, max(s) là giá trị lớn nhất của s
- Tham số sử dụng: Hằng số nén A=87.6

Lượng tử hóa:

- Lượng tử hóa là quá trình xấp xỉ hóa tín hiệu mẫu tương tự dựa vào số mức lượng tử M. Biên độ của tín hiệu từ GTLN đến GTNN được chia làm các khoảng đều nhau gọi là các bước lượng tử. Nếu biên độ mẫu tín hiêu tương tự nằm trong bước lượng tử nào thì sẽ được xấp xỉ hóa theo mức lượng tử đó. Nếu số mẫu lượng tử càng lớn, khả năng thu được chính xác tín hiệu gốc càng cao.
- Mô phỏng: Lượng tử hóa thực hiện là đều, sử dụng hàm quantiz(s,P) trong đó s là tín hiệu mẫu tương tự, P là vecto các mức lượng tử. P có dạng P=min(s):delta:max(s) trong đó delta là bước lượng tử.
- Tham số sử dụng: từ tín hiệu s(t) ta thấy GTLN của tín hiệu là 11 và GTNN của tín hiệu là -11, số mức lượng tử sử dụng là 256 nên vecto P=-11:22/255:11 với bước lượng tử delta=22/255

Mã hóa:

- Là ánh xạ một mức lượng tử thành một từ mã nhị phân. Mã hóa thường kết hợp với bộ A/D, tín hiệu qua bộ mã hóa là các xung vuông có biên độ 0 hoặc 1.
- Mô phỏng: Sử dụng hàm de2bi để chuyển các mức lượng tử sang từ mã nhị phân.

Giải mã

- Ánh xạ từ mã nhị phân 8 bit thành các mức lượng tử có trọng số từ 0=>255. Dựa vào vecto các mức lượng tử Pa để có thể ánh xạ trọng số sang mức biên độ bằng thuật toán S(t)=Pa(C(t)+1) trong đó C(t) là trọng số mức lượng tử, S(t) là tín hiệu thu được.

Giải nén luật A

- Giả sử tín hiệu vào bộ giải nén là s1(t), tín hiệu ra bộ giải nén là s2(t) ta có

$$|s2(t)|=\begin{cases} \frac{(1+\ln(A)).|s1(t)|}{A} \; nếu\; 0 \le |s1(t)| \le \frac{1}{1+\ln(A)} \\ \frac{\exp(|s1(t)|.(1+\ln(A))-1)}{A} \; nếu \frac{1}{1+\ln(A)} \le |s1(t)| \le 1 \end{cases}$$

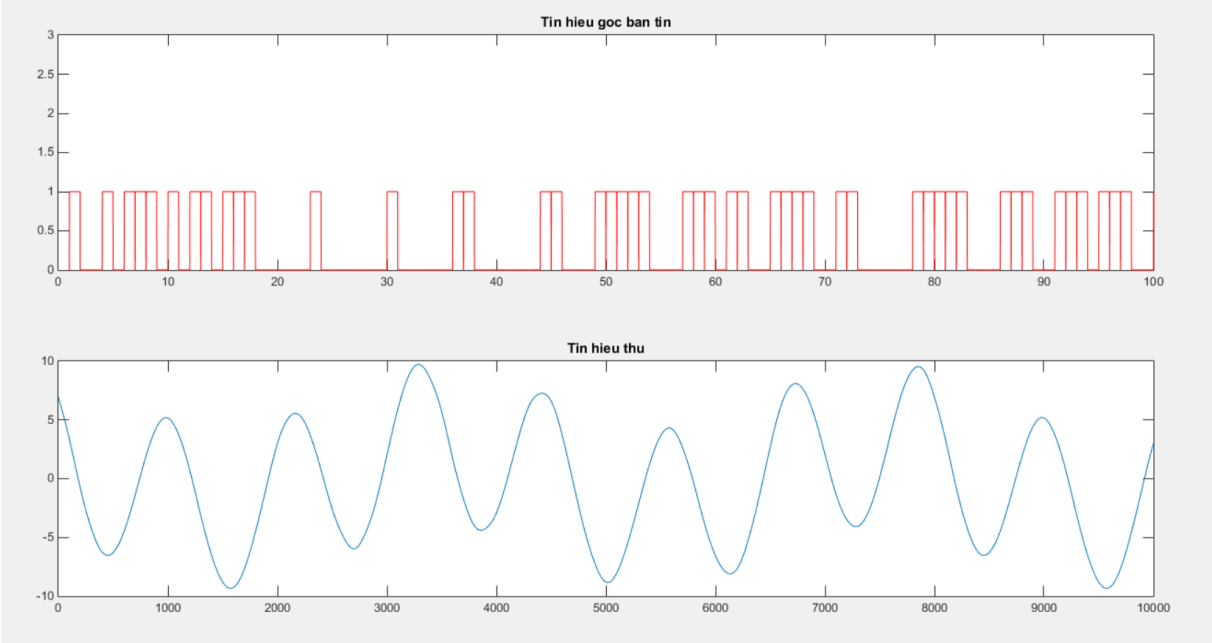
Tái tạo dạng tín hiệu

- Tín hiệu sau bộ giải nén luật A là tín hiệu tương tự ở phía phát. Tuy nhiên đây không phải là tín hiệu gốc ban đầu do sai số ở khâu lượng tử hóa. Để thu được

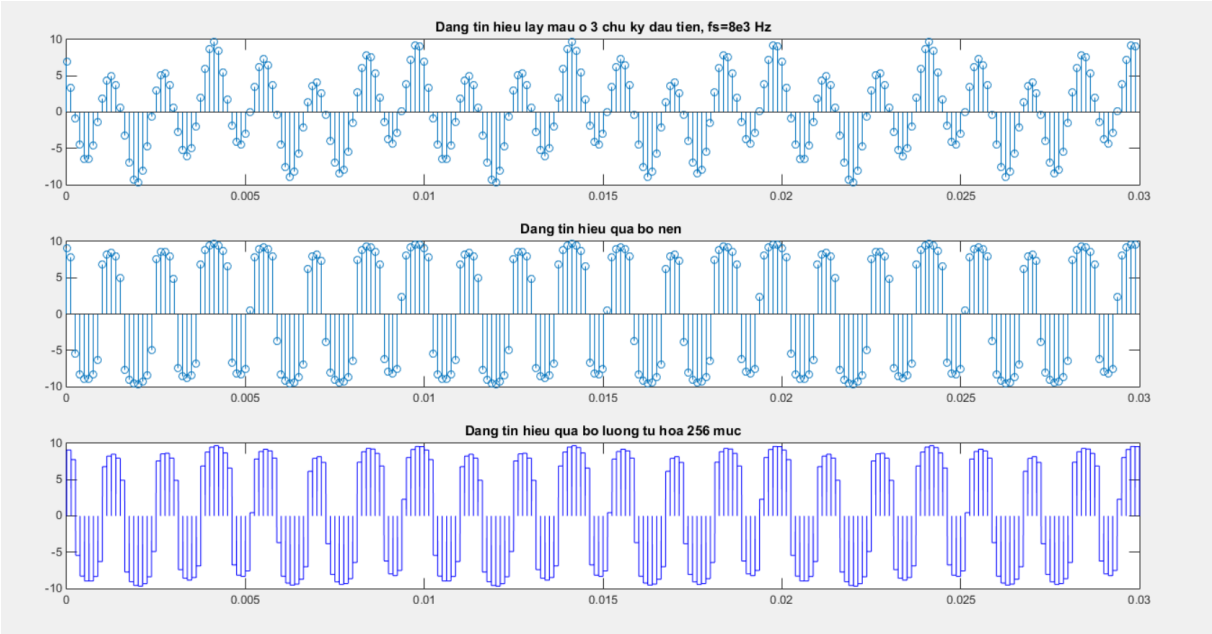
tín hiệu một cách chính xác, nó được đưa qua một bộ nội suy làm tăng số mẫu tín hiệu

- Trong mô phỏng sử dụng hàm interp(x,s) để tăng tần số lấy mẫu trong đó x là tín hiệu cần nội suy, s là tỷ số tần số lấy mẫu mới so với tần số lấy mẫu cũ

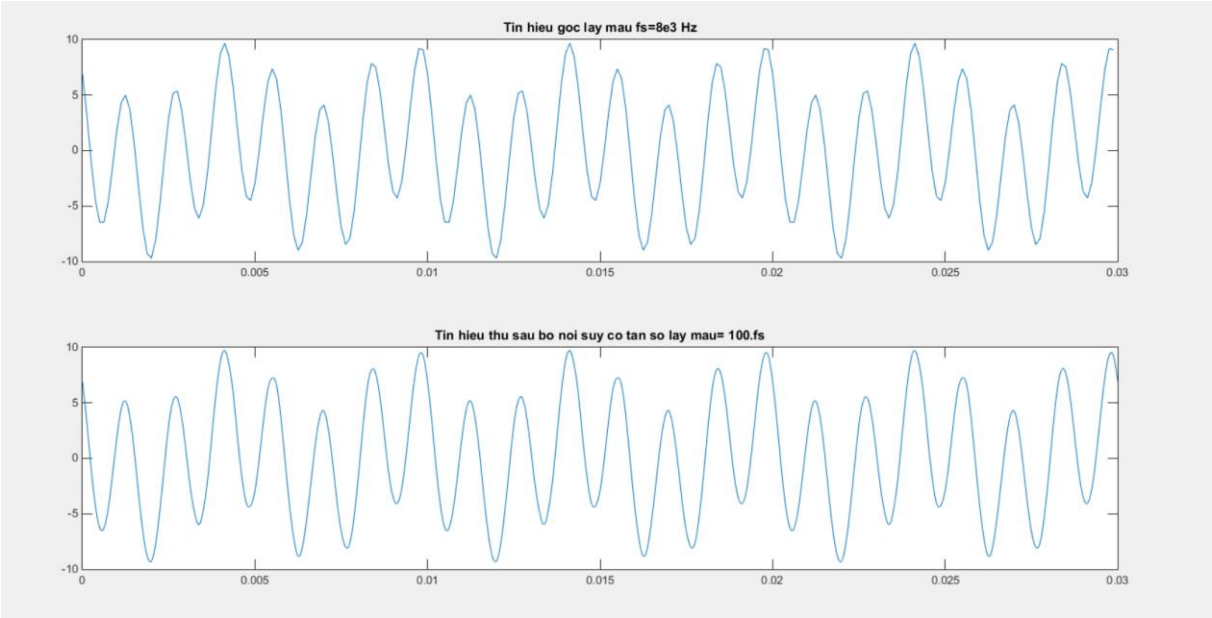
1.3 Kết quả:



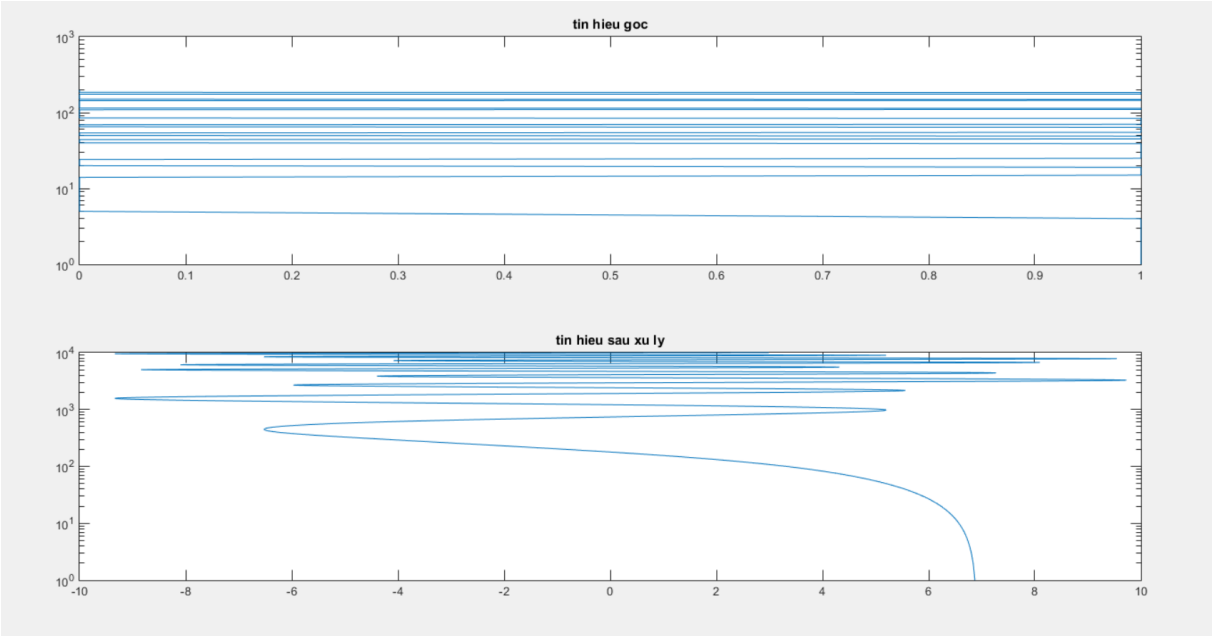
Hình1.2 Tín hiệu của bản tin gốc và thu



Hình1.3 Tín hiệu qua các khâu phía mã hóa



Hình 1.4: Tín hiệu lấy mẫu gốc và tín hiệu giải mã sau xử lý thu được



Hình 1.5: PSD của tín hiệu phát ở dạng xung vuông nhị phân và tín hiệu thu sau xử lý

1.4 Mã chương trình

```
%task1
fs=8e3;          % tan so lay mau
step=1/fs;       % chu ki lay mau
A=87.6;          % hang so nén luat A
t=0:step:1;      % vecto thoi gian- quá trình lay mau
Pa=-11:22/255:11; % vecto cac muc luong tu hóa, bien do lon nhat va
%nho nhat co the cua s(t) là 11 và -11
s=2*cos(2*pi*200*t)+2*cos(2*pi*200*t+pi/2)+7*cos(2*pi*700*t+pi/4);% qua trinh lay mau
sc = compand(s,A,max(s),'A/compressor');        % nen luat A
C=quantiz(sc,Pa);   % luong tu hoa
% qua trinh ma hoa và bien doi tu ma tran sang vector nhi phân
Code=(de2bi(C))';
Code=(Code(:))';%
S=zeros(1,length(C));
% qua trinh giai ma
for k=1:length(C)
S(k)=Pa(C(k)+1);
end
SC = compand(S,A,max(S),'A/expander');%  giai nen luat A
SC=interp(SC,100);% quá trình noi suy, tan so lay mau moi=100.fs
% ve hinh
figure(1)
subplot(2,1,1)
for k=1:100
plot(k:0.01:k+0.99,Code(k)*[0 ones(1,98) 0],'color','r')
hold on
end
title('Tin hieu goc ban tin')
axis([0,100,0,3])
subplot(2,1,2)
plot(SC(1:100*100))
title('Tin hieu thu')
hold on;
figure(2)
subplot(3,1,1)
num_3s=3*(1/100)/step;
```

```
title('Dang tin hieu lay mau o 3 chu ky dau tien, fs=8e3 Hz')
subplot(3,1,2)
stem(t(1:num_3s),sc(1:num_3s))
title('Dang tin hieu qua bo nen ')
subplot(3,1,3)
%
for k=1:num_3s
plot(t(k):step/99:t(k+1),sc(k)*[0 ones(1,98) 0],'color','b')
hold on
end
title('Dang tin hieu qua bo luong tu hoa 256 muc')
figure(3)
subplot(2,1,1)
plot(t(1:num_3s),s(1:num_3s))
title('Tin hieu goc lay mau fs=8e3 Hz')
subplot(2,1,2)
%
plot(step/100:step/100:3*1/100,SC(1:num_3s*100))
title('Tin hieu thu sau bo noi suy co tan so lay mau= 100.fs')
figure(4)
subplot(2,1,1)
one=ones(1,5);in=[];d=1;
for k=1:5:100*1*2
   in(k:k+4)=one*Code(d);d=d+1;
end
Pf = 0:length(in)-1;
f = in;
semilogy(f,Pf)
title('tin hieu goc');
subplot(2,1,2)
Pf1=0:100*100-1;
f1=SC(1:100*100);
semilogy(f1,Pf1)
title('tin hieu sau xu ly')
hold off;
```

3. Nhiệm vụ 3:

1. Mô tả nhiệm vụ:
Mô phỏng hệ thống truyền dẫn số để truyền tín hiệu số thu được từ:
1) Nhiệm vụ 1: Sau khi chuyển đổi PCM từ Nhiệm vụ 1 trong khoảng thời gian 10 s.
2) Nhiệm vụ 2: Mô phỏng hệ thống truyền dẫn số tại tốc độ dữ liệu N Mb/s, giá trị N được xác định bởi số cuối cùng của mã số sinh viên của nhóm trưởng (Nếu số đó là số 0 thì sẽ lựa chọn số liền kề bên cạnh) với chuỗi bít thu được từ Nhiệm vụ 2.
Trong trường hợp không thực hiện được việc lấy nguồn tin từ nhiệm vụ 1 và 2 được hãy thay thế bằng một chuỗi tín hiệu nhị phân ngẫu nhiên tương đương.
Gọi tổng các số cuối cùng trong mã sinh viên của các thành viên trong nhóm là SC và R là số dư của SC chia cho 3 (R=SC mod 3). Dựa vào SC và R, mỗi nhóm sinh viên sẽ thực
hiện một trong các kỹ thuật điều chế sau cho hệ thống mô phỏng:
- Điều chế M-PSK nếu SC là lẻ, trong đó, M = 2^2+R
- Điều chế M-QAM nếu SC là chẵn, trong đó M = 2^2+R
2. Yêu cầu:
a. Mô tả hệ thống mô phỏng bằng sơ đồ khối và xác định các tham số của hệ thống, có thể bao gồm cả các bộ lọc sử dụng để có được bộ thu tối ưu.
b. Bằng việc sử dụng MATLAB, viết chương trình mô phỏng hệ thống truyền dẫn số sử dụng kỹ thuật điều chế đã lựa chọn trên kênh AWGN với nguồn tín hiệu là:
1) Tín hiệu số PCM được thực hiện ở Nhiệm vụ 1 trong khoảng thời gian 10s hoặc
2) Chuỗi bít ở nhiệm vụ 2 với tốc độ N Mbp/s.
Ước tính xác suất lỗi tại các mức tỉ số tín hiệu trên nhiễu SNR (Eb/N0) bằng 5, 8 và 12 dB theo phương pháp Monte Carlo.
c. Biểu diễn biểu đồ chòm sao, dạng sóng tín hiệu, mẫu mắt và phổ của nó tại các điểm sau trên hệ thống: đầu ra bộ điều chế, sau khi truyền qua kênh AWGN tại SNR = 8 dB, sau khi được xử lý và khôi phục tại bộ thu.
d. So sánh dạng sóng tín hiệu tương tự/chuỗi bít nhị phân file ảnh được khôi phục sau khi truyền qua hệ thống mô phỏng tại các mức SNR yêu cầu.

2. Bài làm:

a. Hệ thống mô phỏng được mô tả bằng sơ đồ khối như sau:
   Chuỗi bít vào --> Bộ điều chế M-QAM --> Kênh AWGN --> Bộ thu --> Bộ giải mã M-QAM --> Chuỗi bít ra

   Các tham số của hệ thống:

   Chuỗi bít vào: 100011110101010
   Tốc độ dữ liệu: 7 Mb/s
   Loại điều chế: M-QAM
   Số mức M: 8
   Mức nhiễu SNR: được chỉ định (ví dụ: 5 dB, 8 dB, 12 dB)
   Các bộ lọc: có thể sử dụng bộ lọc truyền qua (pulse shaping) trước bộ điều chế và bộ lọc khôi phục (matched filter) tại bộ thu để tối ưu hóa hiệu suất của hệ thống.

b. Để mô phỏng hệ thống truyền dẫn số với điều chế M-QAM trong MATLAB, có thể làm như sau:

```
% Các thông số của hệ thống
data_rate = 7e6; % Tốc độ dữ liệu (bps)
M = 8; % Số mức của điều chế M-QAM
SNR = [5, 8, 12]; % Các mức tỉ số tín hiệu trên nhiễu (SNR) cần ước tính
num_bits = 1000; % Số bit để truyền dẫn

% Tạo chuỗi bit nguồn
source_bits = randi([0 1], 1, num_bits);

% Điều chế M-QAM
qam_symbols = qammod(source_bits, M);

% Tạo kênh truyền dẫn và kênh nhiễu AWGN
channel = 1; % Kênh truyền dẫn (1: không có kênh, 2: có kênh)
for i = 1:length(SNR)
    % Tạo kênh nhiễu AWGN với tỉ số tín hiệu trên nhiễu (SNR) cho trước
    snr = SNR(i);
    noise_var = 1 / (2 * data_rate * 10^(snr/10));
    noisy_symbols = awgn(qam_symbols, snr, 'measured');

    % Bộ thu
    received_bits = qamdemod(noisy_symbols, M);

    % Tính xác suất lỗi
    num_errors = sum(source_bits ~= received_bits);
    error_rate(i) = num_errors / num_bits;
end

% Hiển thị kết quả xác suất lỗi
disp(['SNR (dB): ', num2str(SNR)]);
disp(['Error Rate: ', num2str(error_rate)]);
```

c. Để biểu diễn biểu đồ chòm sao, dạng sóng tín hiệu, mẫu mắt và phổ của tín hiệu truyền qua hệ thống truyền dẫn số với điều chế M-QAM và qua kênh AWGN, sau đó được xử lý và khôi phục tại bộ thu, ta có thể sử dụng các hàm và công cụ tính toán trong MATLAB.

Code:

```
% Thông số của hệ thống
M = 8; % Số mức của M-QAM
R = 7e6; % Tốc độ dữ liệu (bps)
```

```
T = 1/R; % Chu kỳ ký hiệu (s)
SNR_dB = 8; % Mức SNR (dB) cần mô phỏng

% Tạo chuỗi bít nguồn
bits = [1 0 0 0 1 1 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0]; % Chuỗi bít thu được

% Điều chế M-QAM
symbols = bi2de(reshape(bits, log2(M), length(bits)/log2(M)).', 'left-msb');
modulated_signal = qammod(symbols, M);

% Biểu diễn biểu đồ chòm sao
scatterplot(modulated_signal);
title('Biểu đồ chòm sao của tín hiệu sau khi điều chế M-QAM');

% Truyền qua kênh AWGN
SNR_linear = 10^(SNR_dB/10); % Chuyển đổi SNR từ đơn vị dB sang đơn vị
tuyến tính
noisy_signal = awgn(modulated_signal, SNR_dB, 'measured');

% Biểu diễn dạng sóng tín hiệu
figure;
plot(real(noisy_signal));
hold on;
plot(imag(noisy_signal));
title('Dạng sóng tín hiệu sau khi truyền qua kênh AWGN');
legend('Phần thực', 'Phần ảo');

% Tính toán mẫu mắt
eyediagram(noisy_signal, 2*T);

% Tính toán phổ của tín hiệu
Fs = 1/T; % Tần số lấy mẫu
N = length(noisy_signal); % Số mẫu của tín hiệu
f = linspace(-Fs/2, Fs/2, N); % Tạo vector tần số
spectrum = abs(fftshift(fft(noisy_signal)))^2/N; % Tính phổ của tín hiệu
figure;
plot(f, spectrum);
title('Phổ của tín hiệu sau khi truyền qua kênh AWGN');
xlabel('Tần số (Hz)');
ylabel('Mật độ năng lượng (W/Hz)');
```

d. Để so sánh dạng sóng tín hiệu tương tự và chuỗi bít nhị phân của file ảnh được khôi phục sau khi truyền qua hệ thống mô phỏng với các mức SNR yêu cầu, ta cần thực hiện các bước sau:

Đọc file ảnh và chuyển đổi thành chuỗi bít nhị phân.

Điều chế chuỗi bít nhị phân thành tín hiệu tương tự sử dụng phương pháp điều chế M-QAM với M = 8.

Truyền tín hiệu tương tự qua hệ thống mô phỏng với các mức SNR yêu cầu.

Khôi phục tín hiệu tương tự thành chuỗi bít nhị phân sử dụng phương pháp đồng bộ hóa và giải mã M-QAM.

So sánh chuỗi bít nhị phân khôi phục được với chuỗi bít nhị phân ban đầu của file ảnh để đánh giá chất lượng truyền dẫn.

Code:

```
% Thông số của hệ thống

M = 8; % Số mức của M-QAM

R = 7e6; % Tốc độ dữ liệu (bps)

T = 1/R; % Chu kỳ ký hiệu (s)

SNR_dB = [0 4 8]; % Các mức SNR yêu cầu để so sánh


% Đọc file ảnh và chuyển đổi thành chuỗi bít nhị phân

image_file = 'image.jpg'; % Tên file ảnh

image = imread(image_file);

bits = reshape(de2bi(image), 1, []); % Chuyển đổi thành chuỗi bít nhị phân


% Điều chế M-QAM

symbols = bi2de(reshape(bits, log2(M), length(bits)/log2(M)).', 'left-msb');

modulated_signal = qammod(symbols, M);


% Truyền qua kênh AWGN và khôi phục tín hiệu

bit_errors = zeros(size(SNR_dB));

for i = 1:length(SNR_dB)
```

```
    SNR_linear = 10^(SNR_dB(i)/10);
    noisy_signal = awgn(modulated_signal, SNR_dB(i), 'measured');
    demodulated_signal = qamdemod(noisy_signal, M);
    received_bits = de2bi(demodulated_signal, log2(M), 'left-msb');
    received_bits = reshape(received_bits.', 1, []);
    bit_errors(i) = biterr(bits, received_bits);
end

% Biểu diễn kết quả
figure;
semilogy(SNR_dB, bit_errors, 'o-');
title('Số lỗi bit trên mỗi bit của file ảnh khôi phục được');
xlabel('SNR (dB)');
ylabel('Số lỗi bit trên mỗi bit');
grid on;
legend('Bit errors');
```